LAB 18

Deadlock Simulation & Prevention

Mekanisme circular wait pada sistem konkuren, eliminasi deadly embrace lewat lock ordering deterministik, dan pemulihan deadlock victim via application retry di Go.

#CircularWait #LockOrdering #DeadlockVictim #AppRetry

transfer.go (order-enforced lock)

```
// Urutkan lock berdasarkan ID: memutus siklus first, second :=
acc1, acc2 if acc1.ID > acc2.ID { first, second = acc2, acc1 } //
Kunci selalu mulai dari ID terkecil if err := first.Lock(ctx); err
!= nil { return err } defer first.Unlock() if err :=
second.Lock(ctx); err != nil { return err } defer second.Unlock()
>> PASS: 0 data races detected // Go 1.22
```

AUTHOR: lukman-ss / Lukman

Masalah: Deadlock pada Sistem Konkuren

Kondisi saling menunggu tanpa akhir saat dua proses memperebutkan shared resources.

MOTIVASI

Ketika dua transaksi mengakses entitas yang sama dengan urutan berlawanan, masing-masing menahan kunci yang dibutuhkan pihak lain. Transaksi membeku selamanya (deadly embrace) tanpa campur tangan eksternal.

BATAS LAB

Lab membuktikan simulasi channel Go standar berbatas context timeout. Tidak mengimplementasikan Wait-For Graph internal storage engine dan menggunakan backoff tetap 2ms untuk kebutuhan test deterministik.

Deadlock is a circular dependency. The database aborts one transaction as a victim.

Mengapa Ini Krusial: Dampak Sistemik Deadlock

Deadlock bukan anomali teoritis, melainkan risiko nyata di sistem throughput tinggi.

DAMPAK OPERASIONAL

Koneksi database pool tertahan, latensi p99 meroket, dan transaksi pengguna dibatalkan paksa. Kegagalan penanganan di aplikasi menyebabkan lonjakan HTTP 500 error ke pengguna akhir.

AKAR MASALAH

Banyak pengembang menganggap deadlock sebagai bug database engine, padahal mayoritas penyebabnya adalah ketiadaan koordinasi urutan akses data di lapisan aplikasi.

Unordered data access at the application layer is the primary catalyst for database deadlocks.

Mental Model: Siklus Tunggu (Circular Wait)

Kondisi ke-4 Coffman: siklus tertutup dependensi kepemilikan sumber daya.

TRANSAKSI 1

Memegang lock pada Akun A, lalu mencoba mengakuisisi lock pada Akun B. Proses terblokir menunggu Akun B dibebaskan oleh Transaksi 2.

TRANSAKSI 2

Memegang lock pada Akun B, lalu mencoba mengakuisisi lock pada Akun A. Proses terblokir menunggu Akun A dibebaskan oleh Transaksi 1.

Circular wait transforms isolated locks into an unbreakable systemic freeze.

Database Engine: Wait-For Graph & Deadlock Monitor

Cara mesin database relasional mendeteksi siklus dependensi antar transaksi.

WAIT-FOR GRAPH (WFG)

Database engine memelihara directed graph di memori. Node merepresentasikan transaksi, dan edge merepresentasikan hubungan tunggu (Tx1 -> Tx2).

DEADLOCK MONITOR

Thread latar belakang berjalan periodik memeriksa siklus tertutup pada WFG. Jika siklus ditemukan, database mendeklarasikan status deadlock.

Relational databases identify deadlock by traversing cycle paths in the in-memory Wait-For Graph.

Deadlock Victim: Eksekusi Rollback Sepihak

Pemutusan siklus tunggu secara paksa oleh mesin basis data.

KRITERIA VICTIM

Database memilih transaksi yang paling murah untuk di-rollback (misalnya transaksi dengan log mutasi paling sedikit) atau berbasis konfigurasi prioritas.

KONSEKUENSI ABORT

Semua lock yang dipegang korban dilepas seketika, status di-rollback, dan error spesifik dikembalikan ke client (PostgreSQL 40P01, MSSQL 1205).

The database forcefully breaks circular wait by terminating the least costly transaction.

Arsitektur Simulasi Lab: In-Memory Mutex & Context

Memodelkan row lock dan deadlock detection di Go tanpa dependensi luar.

BUFFERED CHANNEL (MUTEX)

Setiap akun memiliki channel struct bernilai 1. Akuisisi lock mengambil token dari channel; pelepasan lock mengembalikan token ke channel.

CONTEXT DEADLINE

Jika token lock tidak tersedia sebelum context deadline tercapai, Lock mengembalikan ErrDeadlock, merepresentasikan aksi abort oleh deadlock monitor.

Channel selects with context timeouts elegantly model non-blocking lock acquisition and victim aborts.

Model Akun: internal/bank/account.go

Struktur data murni dengan penguncian berbatas waktu context.

STRUKTUR DATA

type Account struct {
ID string
Balance int
ch chan struct{}
}
Channel berkapasitas 1 slot menjamin mutual exclusion.

AKUISISI LOCK

select {
case <-a.ch: return nil
case <-ctx.Done(): return ErrDeadlock
}
Timeout secara deterministik mengindikasikan transaksi korban.

Explicit context timeouts prevent goroutines from hanging indefinitely during lock contention.

Pendekatan Naif: Pemicu Deadlock (TransferNaive)

Akuisisi lock mengikuti urutan parameter caller tanpa koordinasi global.

POLA PENGUNCIAN

Fungsi TransferNaive mengunci akun from terlebih dahulu, menahan lock selama durasi transaksi (delay), kemudian mencoba mengunci akun to.

SKENARIO GAGAL

Goroutine 1: Transfer(A -> B)
Goroutine 2: Transfer(B -> A)
Keduanya menahan lock pertama dan saling menunggu lock kedua hingga context timeout.

Parameter-order locking inherently risks circular wait under bidirectional concurrent load.

Pencegahan Deterministik: Lock Ordering

Mengeliminasi kondisi circular wait dengan mengurutkan penguncian sumber daya.

PRINSIP KERJA

Setiap transaksi membandingkan ID akun dan selalu mengunci akun dengan identifier lebih kecil terlebih dahulu (acc1.ID < acc2.ID), apapun arah transfernya.

HASIL MATEMATIS

Karena urutan penguncian monotonik global, circular wait mustahil terbentuk. Transaksi kedua tertahan di gerbang pertama, mengantre secara tertib.

Lock ordering eliminates the circular wait condition by enforcing global monotonic acquisition.

Implementasi: TransferOrdered

Pengurutan leksikografis ID sebelum akuisisi kunci pertama.

KODE PENGURUTAN

```
first, second := acc1, acc2
if acc1.ID > acc2.ID {
first, second = acc2, acc1
}
```

Sumber daya selalu diurutkan sebelum baris penguncian pertama.

EKSEKUSI AMAN

first.Lock(ctx) lalu second.Lock(ctx).
Kedua transfer bersaing memperebutkan lock pertama yang sama.
Pemenang menyelesaikan mutasi tanpa risiko siklus.

Sorting lock targets before acquisition converts deadlock conflicts into sequential queueing.

Pemulihan Otomatis: Application-Level Retry

Menangani transaksi korban yang dibatalkan oleh database engine.

KEBUTUHAN RETRY

Pada query dinamis kompleks, lock ordering global tidak selalu dapat dipaksakan. Aplikasi harus menangkap error deadlock dan melakukan eksekusi ulang.

STRATEGI BACKOFF

Memberikan jeda waktu singkat sebelum retry memberi kesempatan bagi transaksi pemenang untuk menyelesaikan commit dan melepas kuncinya.

Application retry is mandatory to ensure resilient recovery when deadlock victims are aborted.

Implementasi: TransferWithRetry

Membungkus transfer naif dengan perulangan berbatas waktu context.

LOGIKA RETRY

```
for i := 0; i < maxRetries; i++ {
attemptCtx, cancel := context.WithTimeout(ctx, 10*time.Millisecond)
err := TransferNaive(attemptCtx, ...)
cancel()
if err == nil { return nil }
}
```

PENANGANAN ERROR

Hanya mengulang jika error berupa ErrDeadlock atau DeadlineExceeded. Error validasi domain diteruskan langsung ke pemanggil tanpa retry sia-sia.

Transparent retries absorb transient deadlock errors without escalating failures to users.

Verifikasi Uji 1: Deadlock Terjadi pada Pola Naif

TestDeadlockOccurrence membuktikan pembentukan circular wait.

SETUP PENGUJIAN

Dua goroutine menjalankan TransferNaive bolak-balik (A ke B, dan B ke A) secara konkuren dengan durasi delay 10ms dan context timeout 20ms.

BUKTI ASSERTION

Test memverifikasi bahwa minimal satu transaksi mengembalikan ErrDeadlock (deadlock victim), membuktikan hipotesis kegagalan naif.

Concurrent bidirectional transfers naively executed inevitably produce deadlock victims.

Verifikasi Uji 2: Lock Ordering Mencegah Deadlock

TestLockOrderingPreventsDeadlock membuktikan efektivitas pengurutan.

SETUP PENGUJIAN

Dua goroutine menjalankan TransferOrdered bolak-balik (A ke B, dan B ke A) secara simultan pada kondisi konkurensi dan durasi yang sama.

BUKTI ASSERTION

Kedua goroutine selesai dengan err == nil. Saldo total kedua akun tetap utuh dan konsisten tanpa ada kegagalan transaksi.

Strict lock ordering guarantees zero deadlocks regardless of transaction concurrency levels.

Verifikasi Uji 3: Pemulihan Lewat Application Retry

TestRetryRecoversDeadlock memverifikasi ketahanan pemulihan transaksi.

SETUP PENGUJIAN

Transfer naif dua arah dibungkus TransferWithRetry dengan batas maxRetries = 10 dan delay transaksi 10ms.

BUKTI ASSERTION

Meskipun tabrakan lock awal terjadi, transaksi korban berhasil pulih pada percobaan berikutnya. Kedua transfer berhasil 100%.

Application retries recover aborted transactions, ensuring eventual transactional consistency.

Verifikasi Uji 4: Dampak Durasi Transaksi

TestTransactionDurationImpact membuktikan korelasi durasi terhadap deadlock.

KOMPARASI METRIK

Menguji 20 pasang transfer konkuren pada durasi transaksi panjang (5ms) dibandingkan durasi instan (0ms).

BUKTI EMPIRIS

Frekuensi deadlock pada transaksi panjang terbukti lebih tinggi (longDeadlocks >= shortDeadlocks) karena jendela kompetisi terbuka lebih lama.

Holding locks longer widens the contention window and exponentially raises deadlock probability.

Pencegahan vs Pemulihan: Matriks Strategi

Membandingkan karakteristik operasional Lock Ordering dan Retry.

LOCK ORDERING (PENCEGAHAN)

Kelebihan: 0% deadlock, latensi terprediksi, hemat CPU.
Kekurangan: Menuntut disiplin kode global; sulit pada query ad-hoc kompleks.

RETRY (PEMULIHAN)

Kelebihan: Resilien terhadap kegagalan tak terduga; mudah dipasang.
Kekurangan: Membebani database saat load tinggi; butuh idempotensi ketat.

Prevent deadlocks architecturally with ordering; absorb edge-case anomalies with retries.

Panduan Produksi 1: Universal Ordering & Batch SQL

Aturan pengurutan harus konsisten di seluruh penjuru sistem.

KONSISTENSI GLOBAL

Mengurutkan ID pada endpoint pembayaran tidak berguna bila cron job rekonsiliasi mengakses tabel tanpa pengurutan. Urutan harus seragam.

BATCH MUTASI SQL

Pada operasi multi-baris (UPDATE / SELECT FOR UPDATE), selalu tambahkan ORDER BY id ASC sebelum melakukan klausa penguncian.

A single unsorted query path compromises the deadlock safety of the entire system.

Panduan Produksi 2: Transaksi Singkat & Nol I/O

Meminimalkan waktu penahanan kunci eksklusif di database.

HINDARI NETWORK I/O

Jangan pernah memanggil HTTP API eksternal (payment gateway, notifikasi) di dalam blok transaksi database yang sedang menahan lock baris.

PERSIAPKAN DATA AWAL

Validasi parameter, siapkan payload, dan hitung bisnis rule di memori sebelum membuka blok transaksi `BEGIN ... COMMIT`.

Keep database transactions strictly in-memory and ephemeral to shrink lock windows.

Panduan Produksi 3: Backoff Jitter & Idempotensi

Mencegah fenomena retry storm dan eksekusi ganda pada sistem produksi.

EXPONENTIAL + JITTER

Gunakan backoff eksponensial dengan acakan jitter pada produksi alih-alih fixed backoff, agar ribuan transaksi korban tidak retry bersamaan.

IDEMPOTENSI MUTLAK

Pastikan blok transaksi yang diulang aman dieksekusi berkali-kali tanpa menghasilkan mutasi saldo ganda atau riwayat duplikat.

Combine exponential jittered backoff with strict idempotency to safely execute retries at scale.

Kesalahan Umum Arsitektur (Anti-Patterns)

Pola implementasi yang kerap memicu deadlock di lingkungan produksi.

TRANSAKSI GEMUK (FAT TX)

Menggabungkan logging, audit trail, notifikasi email, dan transfer dana dalam satu transaksi besar yang menahan lock puluhan detik.

PENGABAIAN ERROR KORBAN

Membiarkan error deadlock lolos ke controller API dan diterjemahkan sebagai unhandled 500 error tanpa mekanisme retry otomatis.

Fat transactions and unhandled victim errors turn manageable concurrency into system outages.

Checklist Rilis Rekayasa Sistem Konkuren

Daftar periksa sebelum merilis fitur mutasi multi-sumber daya ke produksi.

PENCEGAHAN & DESAIN

[ ] Seluruh akses baris terurut monotonik (ORDER BY id ASC)
[ ] Nol panggilan HTTP / RPC di dalam blok transaksi aktif
[ ] Durasi transaksi seminimal mungkin (< 20ms)

PEMULIHAN & OPERASIONAL

[ ] Handler error menangkap kode deadlock spesifik database
[ ] Logika retry menggunakan backoff eksponensial + jitter
[ ] Batas maksimum retry ditetapkan (3-5 kali)

Verify monotonic lock ordering and resilient retry handling before deploying concurrent mutations.

SOFTWARE ENGINEERING LAB | LAB 18

Urutkan Penguncian, Pulihkan Korban Transaksi.

Break circular wait with monotonic lock ordering. Recover aborted victims with application retries.

Software Engineering Lab 18 — Deadlock Simulation & Prevention

`internal/bank/account.go` • `internal/transfer/transfer.go` • `tests/transfer_test.go` • `cmd/demo/main.go`

REPOSITORY: github.com/lukman-ss/software-engineering-lab/tree/main/labs/18-deadlock

Lukman / lukman-ss

Verified Go 1.22 (Race-Clean) • 4/4 Tests PASS • commit 6649acb946725e993e78f99ec47833a5498cc870